CS22412 – Artificial Intelligence and Machine Learning Laboratory

EX.NO:1
DATE:

Implement Exhaustive Search for Tic-Tac-Toe Game

AIM:

To implement an exhaustive search algorithm for the Tic-Tac-Toe game using Python to explore all possible game states and determine the optimal outcome.

ALGORITHM:

Step 1: Start.
Step 2: Initialize a 3×3 Tic-Tac-Toe board with empty spaces.
Step 3: Display the board and position indices (0–8).
Step 4: Define a function print_board to display the grid in rows with separators.
Step 5: Define a function check_winner to return 'X' or 'O' if any winning combination matches; else return None.
Step 6: Define a function to check whether the board is full.
Step 7: Implement Exhaustive Search (Minimax):
Base Cases: 'X' wins → +1, 'O' wins → −1, Draw → 0.
Recursive Cases:
Maximizing (AI = X): Try all empty cells → place 'X' → call minimax(False) → undo move → keep maximum score.
Minimizing (Human = O): Try all empty cells → place 'O' → call minimax(True) → undo move → keep minimum score.
Evaluate all empty cells and choose the move with the highest score.
Step 8: Alternate turns between AI and human until a win or draw occurs.
Step 9: After each move, check for a winner or draw condition.
Step 10: If a winner or draw is found, display the result and stop the game.
Step 11: End.

SOURCE CODE:

```
board = [' ' for _ in range(9)]
def print_board():
    print()
    for i in range(3):
        print("|".join(board[i*3:(i+1)*3]))
        if i<2:
            print("-------")
    print()
def check_winner(b):
    wins=[(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),(2,4,6)]
    for a,b1,c in wins:
```

```
        if b[a] == b[b1] == b[c] and b[a]!=' ':
            return b[a]
    return None
def board_full(b):
    return ' 'not in b
def exhaustive_search(b ,player):
    if check_winner(b) or board_full(b):
        return
    for i in range(9):
        if b[i]== ' ':
            b[i]=player
            exhaustive_search(b,'0' if player == 'x' else 'x')
            b[i]= ' '
def ai_move():
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'X'
            exhaustive_search(board,'0')
            return i
print("TIC-TAC-TOE")
print("AI =X | human=0")
print("positions:")
print(" 0 1 2\n3 4 5\n6 7 8")

print_board()

while True:
    move = ai_move()
    board[move] = 'x'
    print("AI played:",move)
    print_board()

    if check_winner(board):
        print("winner:",check_winner(board))
        break
    if board_full(board):
        print("game draw")
        break

    pos=int(input("your move (0-8): "))
    if board[pos] !=' ':
        print("invalid move. try again.")
        continue
    board[pos]='0'
    print_board()

    if check_winner(board):
        print("winner:",check_winner(board))
```

```
        break
    if board_full(board):
        print("game draw")
        break
```

OUTPUT:

```
TIC-TAC-TOE
AI =X | human=0
positions:
 0 1 2
3 4 5
6 7 8

   | |
-------
   | |
-------
   | |

AI played: 0

X | |
-------
   | |
-------
   | |

your move (0-8):  6
x | |
-------
   | |
-------
0 | |

AI played: 1

X |x|
-------
   | |
-------
0  | |

your move (0-8):  2
x |x| 0
-------
   | |
-------
```

```
0 |  |

AI played: 3

X |x| 0
-------
X |  |
-------
0 |  |

your move (0-8):  4
x |x| 0
-------
X |0|
-------
0 |  |

winner: 0
```

RESULT:

The Tic-Tac-Toe game using exhaustive search was implemented and the output is extracted and verified successfully.

EX.NO:2
DATE:

Implement Minimax Algorithm for Tic-Tac-Toe Game

AIM:

To implement the Minimax algorithm for the Tic-Tac-Toe game using Python in order to determine the optimal move for the AI player.

ALGORITHM:

Step 1: Initialize a 3×3 Tic-Tac-Toe board with empty positions.
Step 2: Define all possible winning combinations for the game board.
Step 3: Check whether a player has won or if the board is full to identify terminal states.
Step 4: Implement the Minimax function to evaluate board positions recursively.
Step 5: Assign scores to terminal states: +1 for AI win, −1 for human win, and 0 for draw.
Step 6: Maximize the score for the AI player and minimize the score for the human player.
Step 7: Select the move that yields the best Minimax score for the AI.
Step 8: Repeat the process until the game ends with a win or a draw.

SOURCE CODE:

```
board = [' ' for _ in range(9)]

def print_board():

    print()

    for i in range(3):

        print("|".join(board[i*3:(i+1)*3]))

        if i < 2:

            print("-------")

    print()

def check_winner(b):

    wins = [(0,1,2),(3,4,5),(6,7,8),

            (0,3,6),(1,4,7),(2,5,8),

            (0,4,8),(2,4,6)]
```

```
    for a, b1, c in wins:
        if b[a] == b[b1] == b[c] and b[a] != ' ':
            return b[a]
    return None
def board_full(b):
    return ' ' not in b
def minimax(b, is_maximizing):
    winner = check_winner(b)
    if winner == 'X':
        return 1
    elif winner == 'O':
        return -1
    elif board_full(b):
        return 0
    if is_maximizing:
        best_score = -100
        for i in range(9):
            if b[i] == ' ':
                b[i] = 'X'
                score = minimax(b, False)
                b[i] = ' '
                best_score = max(score, best_score)
        return best_score
    else:
```

```
        best_score = 100
        for i in range(9):
            if b[i] == ' ':
                b[i] = 'O'
                score = minimax(b, True)
                b[i] = ' '
                best_score = min(score, best_score)
        return best_score
def ai_move():
    best_score = -100
    move = 0
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'X'
            score = minimax(board, False)
            board[i] = ' '
            if score > best_score:
                best_score = score
                move = i
    return move
print("TIC-TAC-TOE using MINIMAX")
print("AI = X | Human = O")
print("Positions:")
print("0 1 2\n3 4 5\n6 7 8")
```

```
print_board()
while True:
    move = ai_move()
    board[move] = 'X'
    print("AI played:", move)
    print_board()
    if check_winner(board):
        print("Winner:", check_winner(board))
        break
    if board_full(board):
        print("Game Draw")
        break
    pos = int(input("Your move (0-8): "))
    if board[pos] != ' ':
        print("Invalid move. Try again.")
        continue
    board[pos] = 'O'
    print_board()
    if check_winner(board):
        print("Winner:", check_winner(board))
        break
    if board_full(board):
        print("Game Draw")
        break
```

OUTPUT:

```
TIC-TAC-TOE using MINIMAX

AI = X | Human = O

Positions:

0 1 2

3 4 5

6 7 8

 | |

-------

 | |

-------

 | |

AI played: 0

X| |

-------

 | |

-------

 | |

Your move (0-8): 2


X| |O

-------

 | |

-------
```

| |

AI played: 3

X| |O

X| |

| |

Your move (0-8): 6

X| |O

X| |

O| |

AI played: 4

X| |O

X|X|

O| |

Your move (0-8): 8

X| |O

X|X|

```
-------
O| |O
AI played: 5
X| |O
-------
X|X|X
-------
O| |O
Winner: X
```

RESULT:

The Tic-Tac-Toe game using the Minimax algorithm was implemented and the output is extracted and verified successfully.

EX.NO:3
DATE:

Implement A* Algorithm to find the shortest path

Aim:

To implement the **A*** algorithm to find the shortest path between source and destination using heuristic function.

Algorithm:

Step 1: Start the program.
Step 2: Initialize open list (nodes to visit) and closed list (visited nodes).
Step 3: Add the start node to open list.
Step 4: Assign g(n)=0 and calculate f(n)=g(n)+h(n).
Step 5: Select node with lowest f(n) from open list.
Step 6: If selected node is goal → stop and print path.
Step 7: Otherwise, move node to closed list and expand its neighbors.
Step 8: Update g(n), h(n), f(n) for neighbors and repeat until goal is reached.

Program:

```
def astar(graph, h, start, goal):
    open_list = [start]
    closed = []
    g = {start: 0}
    parent = {start: None}
    while open_list:
        n = min(open_list, key=lambda x: g[x] + h[x])
        if n == goal:
            path = []
            while n:
                path.append(n)
                n = parent[n]
            return path[::-1], g[goal]   # return cost also
        open_list.remove(n)
        closed.append(n)
        for (m, cost) in graph[n]:
            if m not in open_list and m not in closed:
                open_list.append(m)
                parent[m] = n
                g[m] = g[n] + cost
            else:
                if g[m] > g[n] + cost:
                    g[m] = g[n] + cost
```

```
                parent[m] = n
                if m in closed:
                    closed.remove(m)
                    open_list.append(m)
    return None, None
graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 1)],
    'C': [('D', 1)],
    'D': []
}
h = {'A': 4, 'B': 2, 'C': 1, 'D': 0}
path, cost = astar(graph, h, 'A', 'D')
print("Path:", path)
print("Cost:", cost)
```

Output:

```
Path: ['A', 'B', 'D']
Cost: 2
```

Result:

The A* algorithm was successfully implemented and the shortest path along with minimum cost was obtained.

EX.NO : 4
DATE:

Implement BFS and DFS for Water Jug Problem

Aim:

To implement Breadth First Search (BFS) and Depth First Search (DFS) to solve the Water Jug problem and reach the target state.

Algorithm:

Step 1: Start the program.
Step 2: Define jug capacities (A, B) and target value.
Step 3: Represent state as (x, y) where x and y are water in jugs.
Step 4: Initialize initial state (0, 0).
Step 5: For BFS → use queue, for DFS → use stack/recursion.
Step 6: Generate all possible next states:

- Fill A, Fill B
- Empty A, Empty B
- Pour A → B, Pour B → A

Step 7: Check if any state equals target → stop.
Step 8: Otherwise, continue until solution is found or all states are explored.

Program (BFS):

```
from collections import deque
def bfs(a, b, target):
    visited = set()
    q = deque([(0, 0)])
    while q:
        x, y = q.popleft()
        print((x, y))
        if x == target or y == target:
            return "Target Reached"
        if (x, y) in visited:
            continue
        visited.add((x, y))
        q.extend([
            (a, y), (x, b),
            (0, y), (x, 0),
            (max(0, x-(b-y)), min(b, x+y)),
            (min(a, x+y), max(0, y-(a-x)))
        ])
    return "Not Possible"
```

```
print(bfs(4, 3, 2))
```

Output (BFS):

```
(0, 0)
(4, 0)
(0, 3)
(4, 3)
(1, 3)
(1, 0)
(0, 1)
(4, 1)
(2, 3)

Target Reached
```

Program (DFS):

```
def dfs(x, y, a, b, target, visited):
    if (x, y) in visited:
        return False
    print((x, y))
    if x == target or y == target:
        return True
    visited.add((x, y))
    return (dfs(a, y, a, b, target, visited) or
            dfs(x, b, a, b, target, visited) or
            dfs(0, y, a, b, target, visited) or
            dfs(x, 0, a, b, target, visited) or
            dfs(max(0, x-(b-y)), min(b, x+y), a, b, target, visited) or
            dfs(min(a, x+y), max(0, y-(a-x)), a, b, target, visited))
print(dfs(0, 0, 4, 3, 2, set()))
```

Output (DFS):

```
(0, 0)
(4, 0)
(4, 3)
(0, 3)
(3, 0)
(3, 3)
(4, 2)

True
```

Result:

The Water Jug problem was successfully solved using BFS and DFS, and the target state was achieved.

EX.NO: 7A

DATE:

Implementation of Linear Regression for House Price Prediction with Package

Aim:

To implement a Python program to predict house prices using the Linear Regression algorithm.

Program Description:

Linear Regression

Linear Regression is a supervised learning technique used to model the relationship between an input variable and an output variable. It fits a straight line to the given data by minimizing the error between actual and predicted values. The model is represented by the equation

$$y = b_0 + b_1 x$$

where $b_0$is the intercept and $b_1$is the slope. The fit() function is used to train the model by learning the optimal parameters from the training data. The predict() function is then used to estimate output values for new or unseen input data. The model performance is evaluated using Mean Squared Error and R-squared value.

Algorithm:
Step 1: Start.
Step 2: Import the required Python libraries for data handling, visualization, and machine learning.
Step 3: Load the California Housing dataset.
Step 4: Convert the dataset into a suitable data structure for processing.
Step 5: Separate the dataset into input features (X) and target variable (y) representing house prices.
Step 6: Split the dataset into training data (80%) and testing data (20%).
Step 7: Import the Linear Regression algorithm.
Step 8: Initialize the Linear Regression model.
Step 9: Train the model using the training dataset.
Step 10: Predict house prices using the testing dataset.
Step 11: Evaluate the model performance using:
Mean Squared Error (MSE)
R-squared (R^2) score
Step 12: Compare the actual house prices with the predicted house prices.
Step 13: Visualize the comparison using a scatter plot.
Step 14: Display the evaluation results and graph.
Step 15: Stop.

Program:

With Package:

```
from sklearn.datasets import fetch_california_housing from
sklearn.model_selection import train_test_split from
sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error import
pandas as pd
import matplotlib.pyplot as plt import
seaborn as sns

housing = fetch_california_housing(as_frame=True) df =
housing.frame
df['target'] = housing.target

X = df.drop(['target'], axis=1) y =
df['target']
X_train, X_test, y_train, y_test = train_test_split( X,
    y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred) r2 =
r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
print("R2 Score:", r2)

comparison = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred})

plt.figure(figsize=(10, 6))
sns.scatterplot(x='Actual', y='Predicted', data=comparison, alpha=0.5)
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    'r--',
    lw=2
)
```

```
plt.xlabel('Actual Prices ($100k)')
plt.ylabel('Predicted Prices ($100k)') plt.title('Actual
vs Predicted House Prices')
plt.show()
```

Sample Input and Output:

```
Loading data...
Splitting data into training (80%) and testing (20%)...
Training Linear Regression model...
Making predictions on the test set...
--- Model Evaluation ---
Mean Squared Error (MSE): 0.00 R-
squared (R2) Score: 1.0000
Generating visualization...
```

Result:

The Python program for predicting house prices using the Linear Regression algorithm was successfully executed.

EX.NO: 7B
DATE:

Implementation of Linear Regression without Package

Aim:

To develop a Python program that calculates and predicts the relationship between input and output data by fitting a straight line using Simple Linear Regression and the least squares method, without using any pre-built libraries.

Program Description:

Linear Regression

Linear Regression is a statistical method used to model the relationship between an independent variable and a dependent variable. It assumes a linear relationship between the variables and fits a straight line that best represents the data. The model is expressed using the equation

$$y = b_0 + b_1 x$$

where $b_0$is the intercept and $b_1$is the slope of the line. The values of $b_0$and $b_1$are computed using the least squares method to minimize prediction error. This model is widely used for prediction and trend analysis.

Algorithm:

Step 1: Start.
Step 2: Generate a set of random input values and corresponding output values using a linear relationship.
Step 3: Display the generated data points using a scatter plot.
Step 4: Calculate the mean of the input values and output values.
Step 5: Compute the slope (b_1) of the regression line using the least squares formula.
Step 6: Compute the intercept (b_0) of the regression line.
Step 7: Form the regression equation:
$y = b_0 + b_1 x$
Step 8: Predict the output values using the regression equation.
Step 9: Evaluate the model performance by calculating:
Mean Squared Error (MSE)
R-squared (R^2) score
Step 10: Plot the regression line along with the actual data points.
Step 11: Display the evaluation results and graph.
Step 12: Stop.

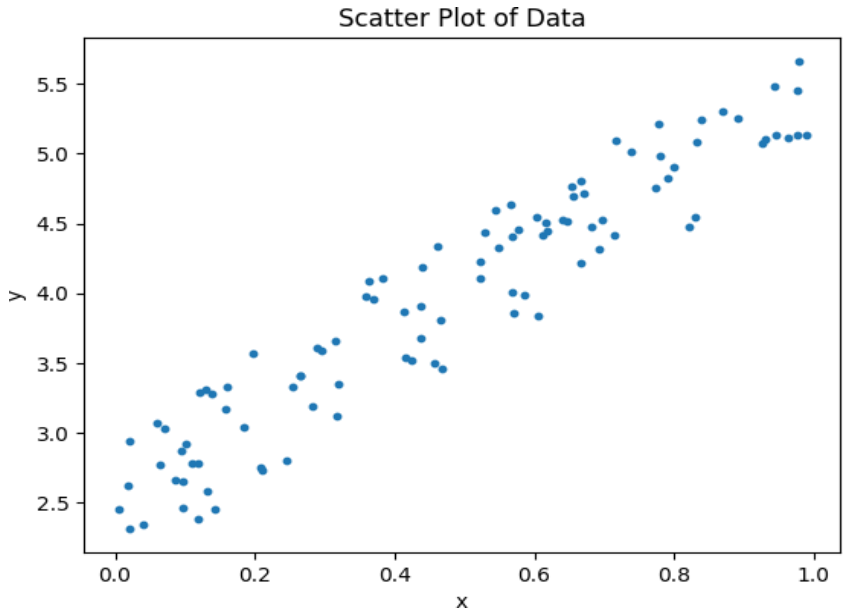
Program:

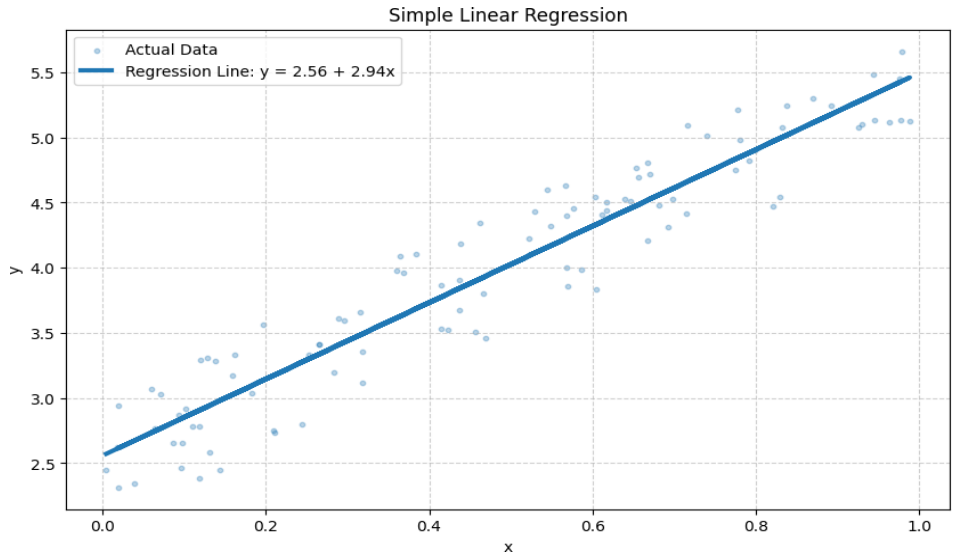
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error, r2_score

np.random.seed(0)

x = np.random.rand(100, 1)
y = 2 + 3 * x + np.random.rand(100, 1)

mean_x = np.mean(x) mean_y =
np.mean(y)

numerator = np.sum((x - mean_x) * (y - mean_y)) denominator
= np.sum((x - mean_x) ** 2)

b1 = numerator / denominator b0 =
mean_y - b1 * mean_x

y_pred = b0 + b1 * x

mse = mean_squared_error(y, y_pred) r2 =
r2_score(y, y_pred)

print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"R-squared Score (R2): {r2:.4f}")

plt.scatter(x, y, s=10)
plt.xlabel('x')
plt.ylabel('y')
plt.title('Scatter Plot of Data')
plt.show()

plt.figure(figsize=(10, 6))
plt.scatter(x, y, alpha=0.3, s=10, label='Actual Data') plt.plot(
    x, y_pred,
    linewidth=3,
    label=f'Regression Line: y = {b0:.2f} + {b1:.2f}x'
)
```

```
plt.xlabel('x')
plt.ylabel('y')
plt.title('Simple Linear Regression')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

Sample Input and Output:

Mean Squared Error (MSE): 0.08
R-squared Score (R2): 0.9039





Result:

The Python program to perform Simple Linear Regression using the least squares method was successfully executed.

EX.NO:8

DATE:

Implementation of Naïve Bayes using Iris Dataset

Aim:

To implement a Python program to classify Iris flower species using the Naive Bayes algorithm and evaluate its performance.

Program Description:

Naive Bayes Classification

Naive Bayes is a supervised machine learning algorithm based on Bayes' Theorem and is widely used for classification tasks. It assumes that all input features are independent, which makes the method simple and efficient.
Bayes' Theorem is given by:

$$P(C \mid X) = \frac{P(X \mid C)\, P(C)}{P(X)}$$

Where:

- $P(C \mid X)$– Posterior probability
- $P(X \mid C)$– Likelihood
- $P(C)$– Prior probability
- $P(X)$– Evidence

Gaussian Naive Bayes is used, which assumes that the features follow a normal distribution. The model is trained using the fit() function and predictions are made using the predict() function. The performance is evaluated using accuracy score and classification report.

Algorithm:

Step 1: Start.
Step 2: Import the required Python libraries for data handling, visualization, and machine learning.
Step 3: Load the Iris dataset from sklearn.datasets.
Step 4: Convert the dataset into a pandas DataFrame for easier analysis.
Step 5: Separate the dataset into:
Input features (X)
Target variable (y) representing flower species
Step 6: Split the dataset into training data (70%) and testing data (30%).
Step 7: Import the Gaussian Naive Bayes classifier.
Step 8: Initialize the Naive Bayes model.

Step 9: Train the model using the training dataset.
Step 10: Visualize feature relationships using a pair plot.
Step 11: Predict the class labels using the testing dataset.
Step 12: Evaluate the model performance using:
Accuracy Score
Classification Report
Step 13: Predict the species for a new sample input.
Step 14: Display the results and graphs.
Step 15: Stop.

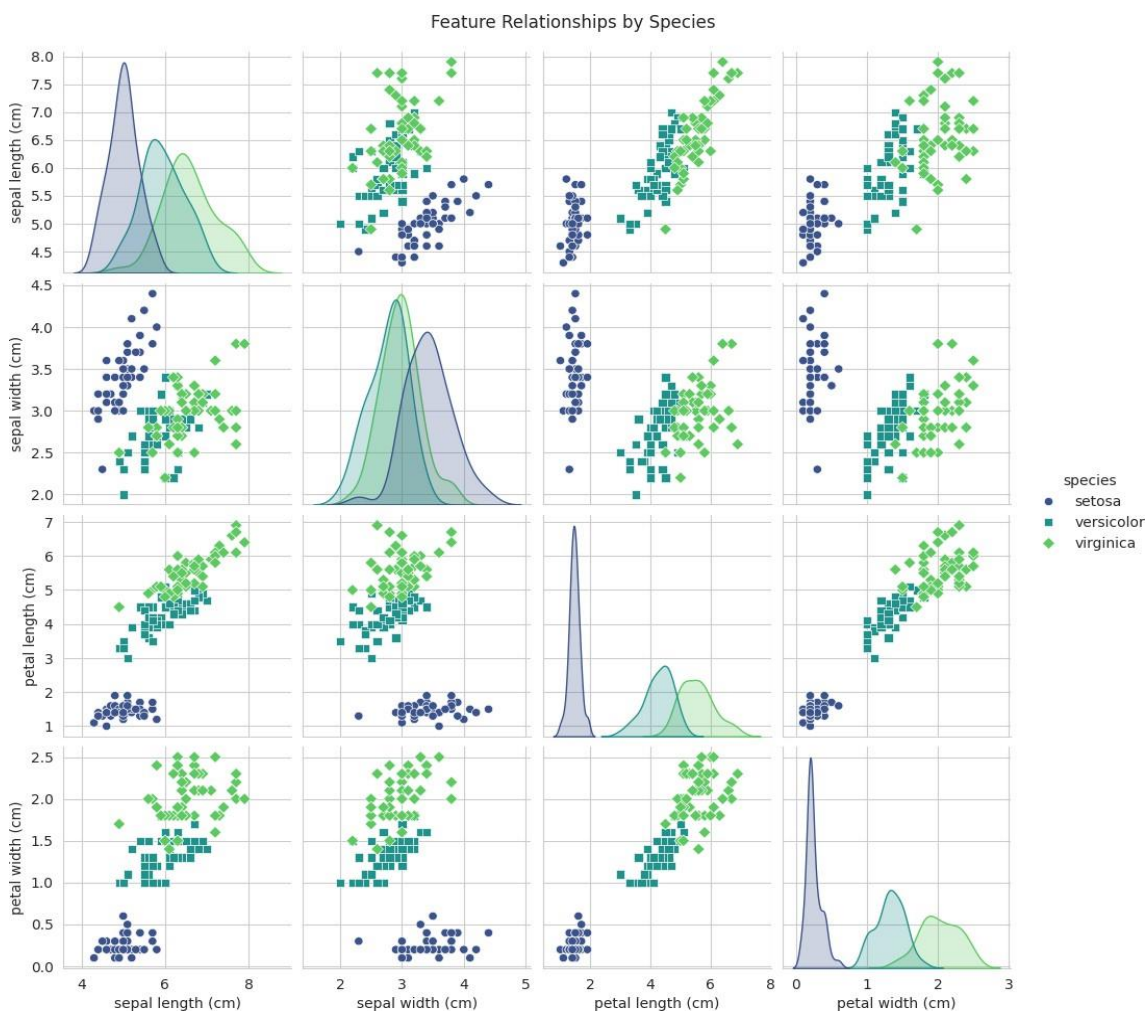
Program:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report

iris = datasets.load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = [iris.target_names[i]
for i in iris.target]

print("First five rows of the Iris dataset:")
print(df.head())
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target, test_size=0.3,
random_state=42)
model = GaussianNB()
model.fit(X_train, y_train)

sns.set_style("whitegrid")
sns.pairplot(df, hue="species", palette="viridis", markers=["o", "s", "D"]) plt.suptitle("Feature
Relationships by Species", y=1.02)
plt.show()

y_pred = model.predict(X_test)

print(f"Model Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

```
new_sample = [[5.1, 3.5, 1.4, 0.2]] prediction
= model.predict(new_sample)
print(f"Predicted Class for sample {new_sample}: {iris.target_names[prediction][0]}")
```

Sample Input And Output:

```
First five rows of the Iris dataset:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

  species
0  setosa
1  setosa
2  setosa
3  setosa
4  setosa
```



Model Accuracy: 0.98

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	19
versicolor	1.00	0.92	0.96	13
virginica	0.93	1.00	0.96	13
accuracy			0.98	45
macro avg	0.98	0.97	0.97	45
weighted avg	0.98	0.98	0.98	45

Predicted Class for sample [[5.1, 3.5, 1.4, 0.2]]: setosa

Result:

The Python program for predicting house prices using the Linear Regression algorithm was successfully executed.

EX.NO:9

DATE:

Implementation of Decision Tree Algorithm using Diabetic Dataset

Aim:

To implement a Python program to classify patients as diabetic or non-diabetic using the Decision Tree algorithm and evaluate its performance using accuracy, precision, recall, F1- score, confusion matrix, and model visualization.

Model Description:

Decision Tree Classifier:

Decision Tree is a supervised machine learning algorithm used for classification. It splits the dataset into smaller parts based on feature values, forming a tree-like structure.

The best feature for splitting is selected using Entropy and Information Gain.

- Entropy(S)=−∑pilog2pi
  Information Gain=Entropy(Parent)−Weighted Entropy(Children)

The model is trained using fit() and predictions are made using predict().

Performance is evaluated using accuracy score, precision, recall, F1-score, confusion matrix, and classification report.

Algorithm:

Step 1: Start.
Step 2: Import the required Python libraries for data handling, visualization, and machine learning.
Step 3: Load the dataset.
Step 4: Convert the dataset into a pandas DataFrame for easier analysis.
Step 5: Split the dataset into training data (75%) and testing data (25%).
Step 6: Import the Decision Tree Classifier.
Step 7: Initialize the Decision Tree model (specify criterion such as entropy or gini).
Step 8: Train the model using the training dataset.
Step 9: Visualize the class distribution using a bar chart.
Step 10: Predict the class labels using the testing dataset.
Step 11: Evaluate the model performance using various metrics.
Step 12: Visualize the trained Decision Tree.
Step 13: Display the results and graphs.
Step 14: Stop.

Program:

```
import pandas as pd
import matplotlib.Pyplot as plt
import seaborn as sns
import os
import kagglehub
from sklearn.model_selection import train_test_split   from
sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score,
f1_score, precision_score, recall_score

path = kagglehub.dataset_download("mathchi/diabetes-data-set") csv_path
= os.path.join(path, "diabetes.csv")
df = pd.read_csv(csv_path)

print("First 5 Rows of Dataset:")
print(df.head())
plt.figure(figsize=(8, 5))
counts = df['Outcome'].value_counts() categories
= ['Non-Diabetic (0)', 'Diabetic (1)'] values =
[counts[0], counts[1]]
plt.bar(categories, values, color=['skyblue', 'salmon'])
plt.title('Distribution of Diabetic vs Non-Diabetic')
plt.xlabel('Diagnosis')
plt.ylabel('Number of Patients')
plt.show()

X = df.drop('Outcome', axis=1) y =
df['Outcome']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
clf = DecisionTreeClassifier(criterion='entropy', max_depth=3, random_state=42) clf.fit(X_train,
y_train)

y_pred = clf.predict(X_test)

print("--- Evaluation Metrics ---")
print(f"Accuracy:  {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision: {precision_score(y_test, y_pred):.4f}")

print(f"Recall:  {recall_score(y_test, y_pred):.4f}")
print(f"F1 Score:  {f1_score(y_test, y_pred):.4f}")
```

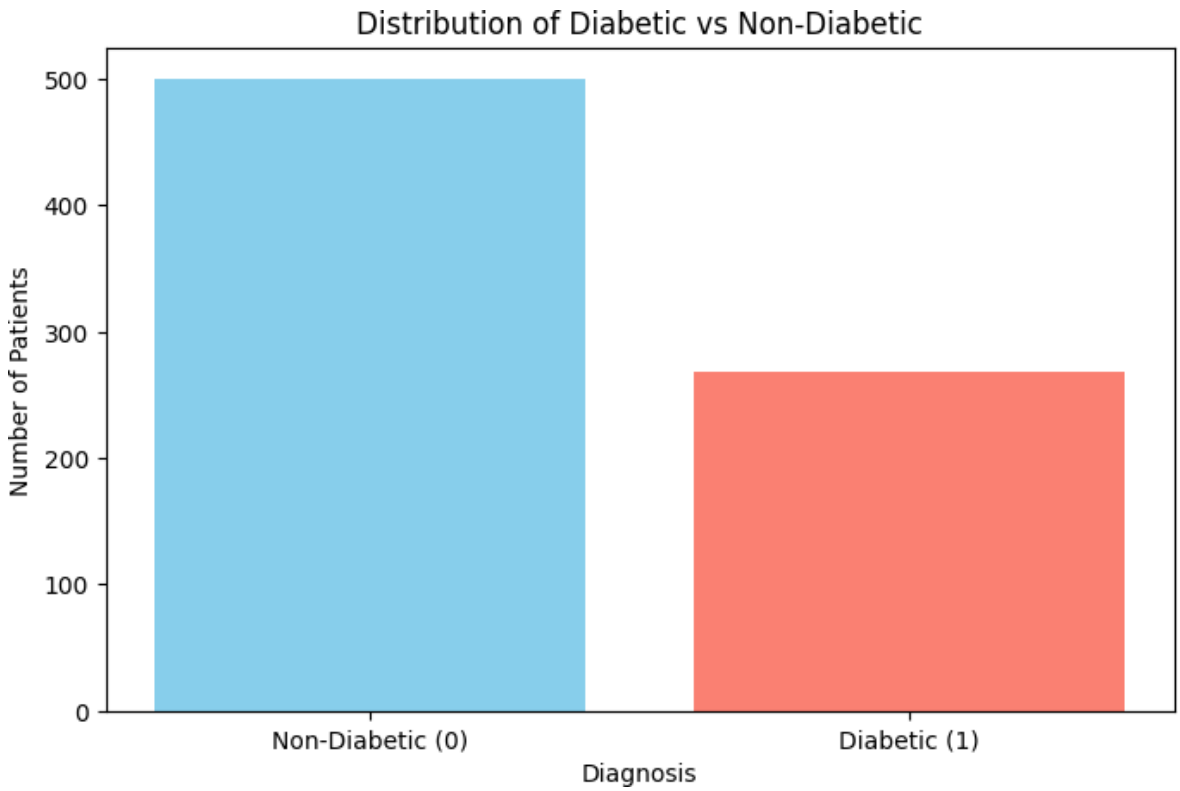
```
print("\n--- Detailed Classification Report ---")
print(classification_report(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues') plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
plt.figure(figsize=(20, 10))
plot_tree(clf,
          feature_names=X.columns,
          class_names=['Non-Diabetic', 'Diabetic'],
          filled=True,
          rounded=True,
          fontsize=12)
plt.title("Decision Tree Visualization") plt.show()
```

Sample Input and Output:

First 5 Rows of Dataset:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	\
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	

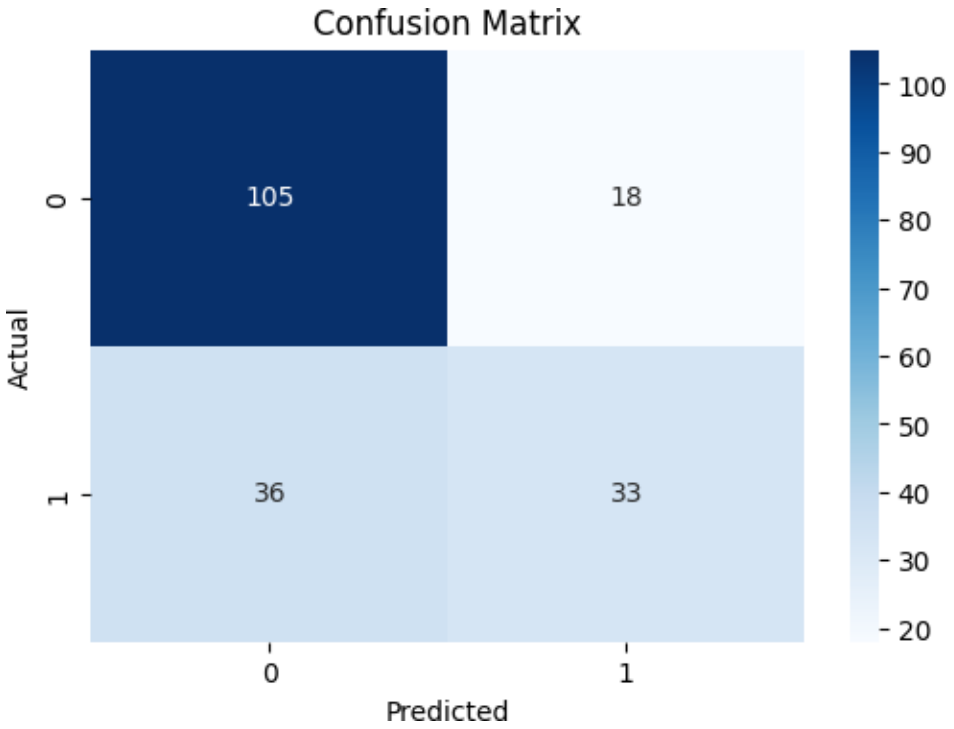
	DiabetesPedigreeFunction	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1

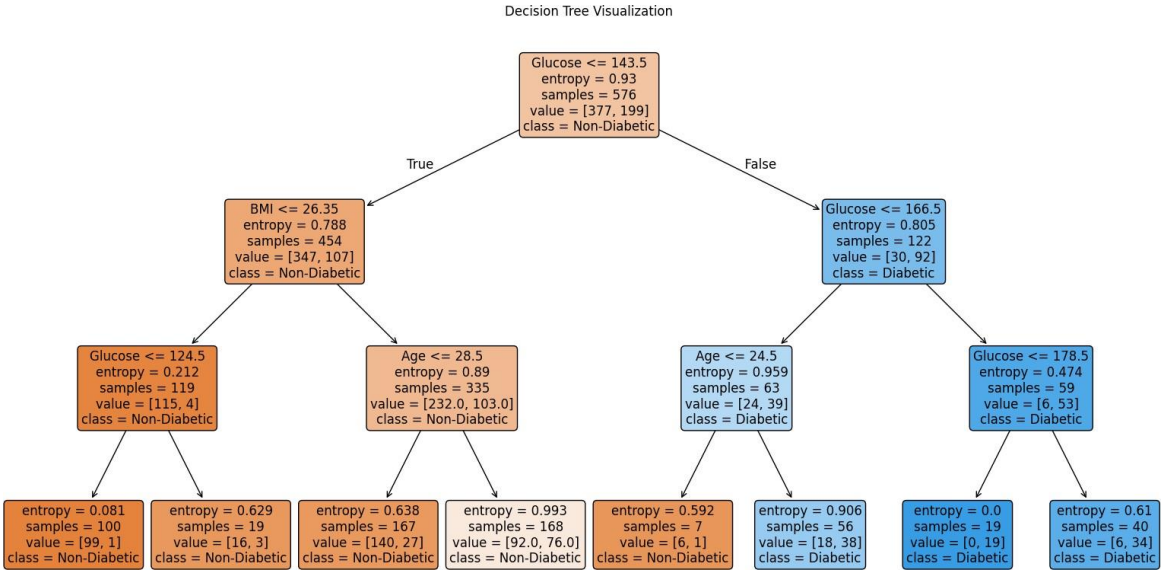


--- Evaluation Metrics --- Accuracy:
0.7188
Precision: 0.6471
Recall: 0.4783
F1 Score: 0.5500

--- Detailed Classification Report ---

	precision	recall	f1-score	support
0	0.74	0.85	0.80	123
1	0.65	0.48	0.55	69
accuracy			0.72	192
macro avg	0.70	0.67	0.67	192
weighted avg	0.71	0.72	0.71	192





Result:

The Python program for implementing Decision Tree Classification was executed successfully. The model was trained and tested, and its performance was evaluated using accuracy score, precision, recall, F1-score, confusion matrix, and classification report.

EX.NO:10

DATE:

Implementation of Multilayer Perceptron model to classify a set of documents

Aim:

To implement a Python program to classify 20 Newsgroups dataset using the Multilayer Perceptron model and evaluate its performance using accuracy, precision, and recall.

Algorithm:

Step 1: Start.
Step 2: Import the required Python libraries for data handling, text vectorization, machine learning model (MLPClassifier), evaluation metrics, and visualization (matplotlib, seaborn, wordcloud).
Step 3: Load the dataset.
Step 4: Generate Word Cloud.
Step 5: Apply TF-IDF vectorization to the text data to convert it into numerical features.
Step 6: Reduce dimensionality using Principal Component Analysis (PCA) to 2 components and visualize using a scatter plot.
Step 7: Split the TF-IDF vectorized dataset into training data and testing data for model training and evaluation.
Step 8: Initialize and train a Multilayer Perceptron (MLP) classifier using the training data.
Step 9: Use the trained MLP classifier to predict the categories for the test dataset.
Step 10: Evaluate and visualize the model performance using various metrics.
Step 11: Stop.

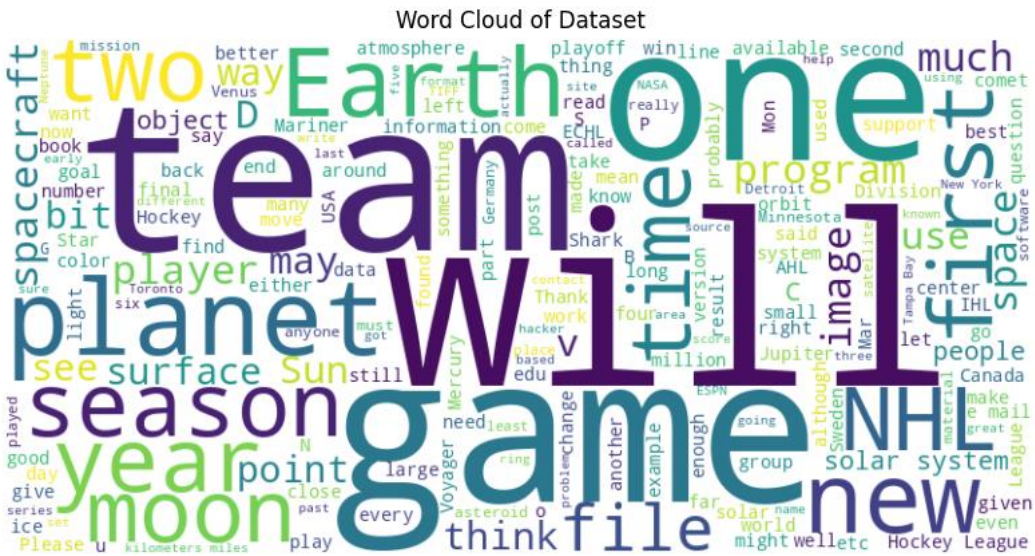
Program:

```
import numpy as np
from sklearn.datasets import fetch_20newsgroups
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import accuracy_score, precision_score, recall_score, classification_report
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud
from sklearn.decomposition import PCA
import numpy as np
categories = ['sci.space', 'rec.sport.hockey', 'comp.graphics']

newsgroups = fetch_20newsgroups(subset='all',categories=categories,
remove=('headers','footers','quotes'))
```

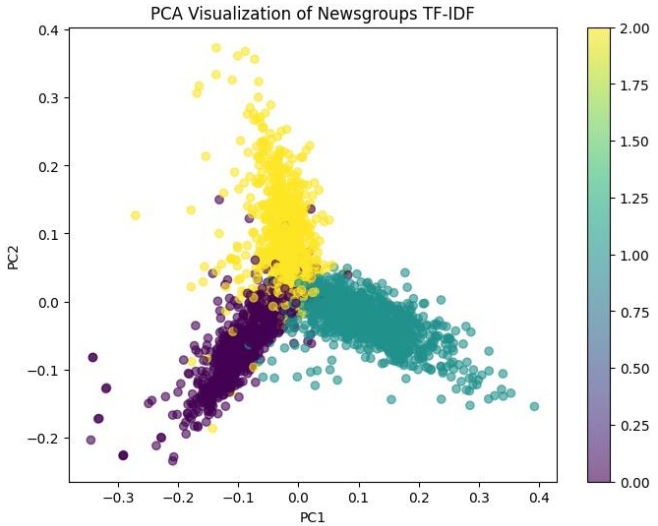
```
X = newsgroups.data

text = " ".join(X[:200])
wordcloud = WordCloud(width=800, height=400,background_color='white').generate(text)

plt.figure(figsize=(10,5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis("off")
plt.title("Word Cloud of Dataset")
plt.show()
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
y = newsgroups.target
vectorizer = TfidfVectorizer(stop_words='english', max_features=5000)
X_vectorized = vectorizer.fit_transform(X)
print(X_vectorized)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_vectorized.toarray())
plt.figure(figsize=(8,6))
plt.scatter(X_pca[:,0], X_pca[:,1], c=y, cmap='viridis', alpha=0.6)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA Visualization of Newsgroups TF-IDF")
plt.colorbar()
plt.show()
X_train, X_test, y_train, y_test = train_test_split( X_vectorized, y, test_size=0.3,
random_state=42)

mlp = MLPClassifier(hidden_layer_sizes=(100,50), activation='relu',solver='adam',
max_iter=20,random_state=1)
mlp.fit(X_train, y_train)

y_pred = mlp.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

precision = precision_score(y_test, y_pred, average='macro')
print("Precision:", precision)

recall = recall_score(y_test, y_pred, average='macro')
print("Recall:", recall)

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Sample Input and Output:


Word Cloud of Dataset
team
Will
game
one
Earth
two
planet
season
year
moon
new
first
time
image
program
spacecraft
player
surface
Sun
point
file
think
NHL
solar system
people
space
use
much
object
bit
may
see
way
Mariner
information
atmosphere
playoff
available
second
comet
support
best
Division
Minnesota
Detroit
New York
orbit
said
system
AHL
center
IHL
Canada
Hockey League
Shark
Hockey
back
end
around
goal
number
final
find
color
Star
light
Toronto
either
data
Thank
work
four
million
Jupiter
right
small
let
go
make
given
even
well
might
world
solar
far
every
play
asteroid
Voyager
Mercury
need
least
large
close
past
good
day
give
ice
Please
kilometers
miles
series
played
group
going
enough
example
another
change
problem
Sweden
although
material
League
e mail
great
Mon
probably
used
read
S
ECHL
take
mean
know
made
post
Germany
part
found
something
USA
move
many
different
Venus
Neptune
mission
better
thing
win
line
help
using
question
software
satellite
Tampa Bay
Mar
three
ESPN
source
result
version
long
B
C
contact
hacker
place
based
edu
anyone
must
got
area
score
really
NASA
P
called
write
last
five
format
TIFF
actually
site
left
come
say
book
now
want
early
move
N
ring
name
set
u
kilometers
miles
Please
series
ice
day
give
good
played
sure
six
G
Star
color
find
number
goal
back
Hockey
final
different
many
move
USA
Germany
post
part
found
something
probably
used
support
best

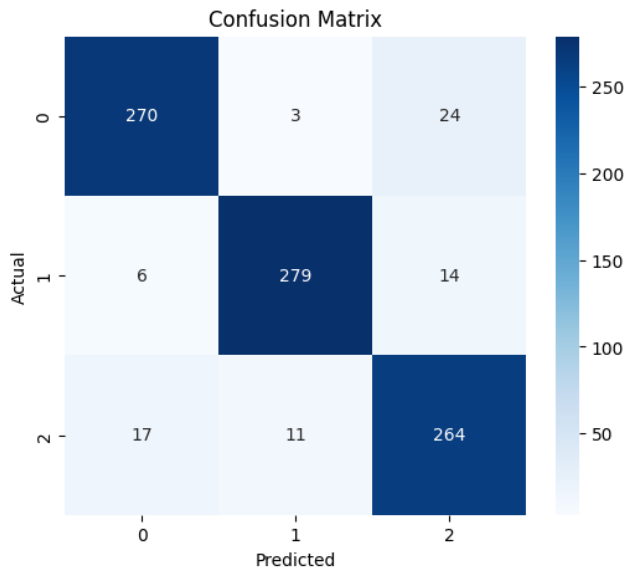
```
(0, 2223)       0.13313825598709797
(0, 2686)       0.08402240368557752
(0, 4747)       0.07765063990192411
(0, 498)        0.10687149195903066
(0, 860)        0.13195868537083746
(0, 3689)       0.13570789808613837
(0, 1853)       0.12199370515443718
:       :
(2957, 4664) 0.21303968313192512
(2957, 2677) 0.13271636134491432
(2957, 588)  0.23157760182751982
(2957, 2771) 0.3889786041001336
(2957, 1615) 0.31607462001852993
(2957, 3209) 0.3007883810696037
(2957, 1827) 0.3213926422689203
(2957, 3853) 0.3047261730936515
(2957, 575)  0.2353173471218569
(2957, 1658) 0.26688304788289885
(2957, 597)  0.32426555143770563
(2958, 4422) 0.4045203732678013
(2958, 4425) 0.18129888936966138
(2958, 4806) 0.5467961522400957
(2958, 883)  0.19995744946893249
(2958, 212)  0.3443736320095303
(2958, 2947) 0.32030001096607064
(2958, 4807) 0.16809904213492938
(2958, 777)  0.21184961016357137
(2958, 1094) 0.16684071420170205
(2958, 397)  0.130417527786418
(2958, 925)  0.15866790669074368
(2958, 1)    0.15771522094012685
(2958, 3843) 0.17443542517202557
(2958, 314)  0.21184961016357137
```



Accuracy: 0.9155405405405406
Precision: 0.9159641073164116
Recall: 0.9154369553416606

Classification Report:

	precision	recall	f1-score	support
0	0.92	0.91	0.92	297
1	0.95	0.93	0.94	299
2	0.87	0.90	0.89	292
accuracy			0.92	888
macro avg	0.92	0.92	0.92	888
weighted avg	0.92	0.92	0.92	888



Result:

The Python program for implementing Multilayer Perceptron model was executed successfully. The model was trained and tested, and its performance was evaluated using accuracyscore, precision and recall.